

DEVOPS MASTER COURSE

Volume 8F

Google Cloud Enterprise Architecture,
Projects & Interview Mastery

Chapters 56-70

Architecture judgment, migration, modernization, SRE, security response, FinOps, platform engineering, certification, interviews, and final enterprise capstone

2026 Edition

DESIGN Make trade-offs explicit	OPERATE Prove resilience with evidence	LEAD Communicate decisions under pressure
---	--	---

About This Part

Volume 8F is the final Google Cloud section of the DevOps Master Course. It converts product knowledge into senior engineering judgment. The emphasis is on architecture decisions, verified operations, measurable reliability, secure migration, incident leadership, cost accountability, and portfolio evidence that can withstand technical review.

The work is deliberately scenario-driven. Every chapter asks you to define requirements, compare alternatives, select a design, implement a controlled exercise, collect evidence, explain residual risk, and improve the operating model. Completed labs should become diagrams, architecture decision records, infrastructure-as-code repositories, runbooks, dashboards, test reports, and concise portfolio case studies.

Graduation principle: a production claim is credible only when another engineer can reproduce the evidence, observe the result, understand the trade-off, and execute the recovery procedure.

Safety, Security, and Cost Controls

- Run migration, failure injection, load testing, security simulation, and regional recovery exercises only in approved nonproduction projects with documented blast radius and stop conditions.
- Use synthetic data, short-lived federation, least-privilege identities, private connectivity where justified, and separate evidence storage. Never expose credentials, customer data, or sensitive logs in repositories or screenshots.
- Create budgets, quotas, expiry labels, cleanup owners, log-retention limits, and automated shutdown before deploying multi-region, performance, or data-replication labs.
- Define RTO, RPO, service-level objectives, rollback triggers, business owner, communications owner, and evidence custodian before any destructive or traffic-changing test.
- Validate current regional availability, quotas, product lifecycle, pricing, certification exam guides, release notes, and support boundaries in official Google Cloud documentation before production use.

Volume 8 Progress

Part	Coverage	Status
8A	Google Cloud foundations, IAM, Compute Engine, VPC, billing	Complete
8B	Core infrastructure, storage, connectivity, backup, DR	Complete
8C	Databases, analytics, data engineering, migration	Complete
8D	GKE, Cloud Run, APIs, messaging, events, workflows	Complete
8E	IaC, CI/CD, observability, security, governance, FinOps	Complete
8F	Enterprise architecture, advanced operations, certifications, interviews	This book

Table of Contents

- Chapter 56 Architecture Decisions, Trade-offs, and Well-Architected Reviews
- Chapter 57 Enterprise SaaS and AI-Enabled Platform Architecture
- Chapter 58 Hybrid Cloud, Distributed Cloud, and Migration Factories
- Chapter 59 Application Modernization and Monolith Decomposition
- Chapter 60 Multi-Region and Global Application Design
- Chapter 61 Security Incident Response and Cloud Forensics
- Chapter 62 Resilience Engineering, Game Days, and Chaos Experiments
- Chapter 63 Performance Engineering and Capacity Planning
- Chapter 64 FinOps and Enterprise Cost Optimization
- Chapter 65 Platform Engineering and Internal Developer Platforms
- Chapter 66 Advanced Troubleshooting and Incident Command
- Chapter 67 Architecture Governance, Compliance, and Evidence
- Chapter 68 Google Cloud Certification Mastery
- Chapter 69 Google Cloud Technical Interview and System Design Mastery
- Chapter 70 Final Integrated Enterprise Google Cloud Capstone
- Part F Command and Investigation Reference
- Production Readiness Graduation Checklist
- Ten Additional Enterprise Portfolio Projects
- 90-Day Google Cloud Engineering Practice Plan
- Glossary
- Official Google Cloud Documentation Index

Chapter 56 - Architecture Decisions, Trade-offs, and Well-Architected Reviews

Turning business goals into measurable, reviewable, evidence-backed Google Cloud decisions

Learning Objectives

- Translate business outcomes into functional requirements, constraints, and quality-attribute scenarios.
- Apply the Google Cloud Well-Architected Framework across operational excellence, security, reliability, performance, cost, and sustainability.
- Create architecture decision records and distinguish reversible experiments from high-consequence commitments.
- Run cross-functional architecture reviews that produce prioritized actions and accepted-risk statements.

56.1 Requirements and Quality Attributes

Architecture begins with outcomes, not products. Convert statements such as “the platform must be highly available” into a quality-attribute scenario that names the source of a stimulus, the stimulus, the environment, the affected component, the expected response, and the response measure. A useful requirement might state that when one serving region becomes unavailable during peak load, authenticated requests continue through another region with no data loss beyond the agreed RPO and with recovery completed inside the RTO.

Capture regulatory boundaries, latency targets, growth assumptions, data residency, identity model, release frequency, support hours, recovery expectations, cost envelope, sustainability goals, and team capability. Unknowns become explicit assumptions with owners and validation dates rather than hidden facts.

56.2 Well-Architected Trade-offs

The Google Cloud Well-Architected Framework provides a common review language. A design can improve one pillar while creating pressure elsewhere: multi-region replication can improve availability but increase cost, operational complexity, and data-consistency risk; strict data perimeters can reduce exfiltration risk but complicate integration and support.

Prioritize risks by business impact and uncertainty rather than attempting to score every recommendation equally. Evidence should include service-level indicators, error budgets, deployment history, audit findings, restore tests, security detections, cost reports, capacity tests, and game-day outcomes.

56.3 Architecture Decision Records

An architecture decision record captures context, decision, alternatives, consequences, status, owner, and review trigger. ADRs preserve organizational memory and prevent teams from repeatedly reopening old choices without new evidence.

High-quality ADRs identify the decision boundary and explicitly describe what the decision does not solve. For example, choosing Cloud Run for a stateless API does not decide the database consistency model, regional topology, queue semantics, or customer identity architecture.

56.4 Review Process and Decision Quality

Architecture review is a risk-discovery mechanism, not a diagram approval meeting. Include application, data, security, networking, operations, reliability, finance, compliance, and business representatives. Review assumptions, failure modes, operational ownership, migration path, rollback, and evidence quality.

Review outcomes should be prioritized findings with owners and dates, not a long undifferentiated checklist. Re-run reviews after significant traffic, regulatory, dependency, architecture, or operating-model changes.

Production principle: prefer decisions that are measurable, reversible where possible, observable in operation, and explainable to the engineers who must recover the system.

Production Design Checklist

- Business outcomes and constraints are measurable.
- Architecture risks are prioritized by impact and uncertainty.
- ADRs record alternatives, consequences, and review triggers.
- Evidence links can be reproduced by an independent reviewer.
- Residual risk has an accountable business owner.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Review becomes a product checklist	Requirements and risks were not defined first	Return to business outcomes and quality attributes	A prioritized risk register replaces service-by-service scoring
Teams disagree without closure	Decision authority and evaluation criteria are unclear	Create ADR owner, deadline, criteria, and escalation path	Decision and dissent are recorded with consequences
Architecture appears healthy but incidents repeat	Evidence relies on configuration, not operating outcomes	Add SLO, incident, restore, deployment, and cost evidence	Review findings map to measurable production behavior
Review actions remain open	Owners and due dates are missing	Assign accountable owner and track through governance cadence	High-risk items close or become formally accepted risk

Hands-on Lab 56: Run a Google Cloud Well-Architected Review

Prerequisites

- A nonproduction workload with diagram, IaC, telemetry, cost data, recovery runbooks, and recent change history.

Implementation

1. Define five business outcomes and convert them into measurable quality-attribute scenarios.
2. Map the workload to the six Well-Architected pillars and identify the three most material cross-pillar trade-offs.
3. Collect evidence for identity, networking, deployment, observability, data protection, cost, and recovery.
4. Create three ADRs for high-consequence choices and one experiment for a reversible choice.
5. Prioritize the top ten risks by impact, likelihood, uncertainty, and recovery difficulty.
6. Present findings to a cross-functional review group and record dissent, accepted risk, owners, and dates.
7. Schedule a follow-up review triggered by a measurable architecture or workload change.

Validation Evidence

- Requirements contain response measures.
- Evidence links are reproducible.
- Findings have owners and dates.
- Accepted risk names the accountable owner.

Cleanup

- Remove temporary review permissions and synthetic evidence data.
- Archive the review package in the approved evidence repository.

Interview Questions and Model Answers

1. What makes an architecture requirement testable?

A defined stimulus, operating condition, expected response, and measurable response criterion.

2. Why record rejected alternatives?

They explain the decision boundary and prevent old debates from restarting without new evidence.

3. What is a reversible decision?

A choice that can be changed with limited cost, downtime, and data impact.

4. How should review findings be prioritized?

By business impact, likelihood, uncertainty, detectability, recovery difficulty, and dependency breadth.

Chapter Summary

- Architecture is a decision system, not a product catalog.
- Use measurable quality attributes and operational evidence.
- Record trade-offs and accepted risk in ADRs.
- Reassess after meaningful change.

Chapter 57 - Enterprise SaaS and AI-Enabled Platform Architecture

Designing secure, tenant-aware, globally scalable software platforms on Google Cloud

Learning Objectives

- Choose tenant-isolation models based on risk, scale, customization, and cost.
- Design tenant-aware identity, routing, data partitioning, quotas, metering, and lifecycle automation.
- Use cells, rings, feature flags, and tenant-level telemetry to limit blast radius.
- Integrate generative AI or agent capabilities without weakening data isolation, privacy, or cost controls.

57.1 Tenant Isolation and Cell Architecture

SaaS isolation is a spectrum. Shared infrastructure maximizes utilization; dedicated projects, clusters, databases, or encryption boundaries maximize isolation. Many enterprises combine models by tenant tier, jurisdiction, workload sensitivity, and support requirements.

A cell architecture groups a bounded set of tenants with independent serving, data, and operational components. Cells reduce correlated failure and enable progressive rollout. A control plane handles tenant placement and lifecycle, while data-plane cells continue operating during control-plane degradation.

57.2 Identity, Routing, and Data Boundaries

Tenant identity must originate from a trusted identity provider and be carried through every authorization decision. Never accept tenant identity solely from an untrusted request field. Use tenant-aware routing at the load balancer, API layer, or application gateway, then enforce authorization again inside the workload.

Partition relational, document, analytical, vector, and object data with explicit tenant keys and verified query patterns. High-assurance tiers may justify dedicated projects, encryption keys, databases, or regions. Audit logs and telemetry must include tenant context without leaking sensitive payloads.

57.3 Provisioning, Metering, and Lifecycle

Tenant onboarding should be idempotent: validate entitlement, select isolation tier and region, provision through versioned automation, configure identity and secrets, create quotas and dashboards, run synthetic tests, and emit a durable completion event. Offboarding must support suspend, export, legal hold, retention, deletion, and evidence of completion.

Metering events require stable identifiers, deduplication, reconciliation, late-event handling, and a clear distinction between usage telemetry and financial billing records. Build cost allocation by tenant, cell, environment, and product capability.

57.4 AI and Agent Guardrails

AI-enabled SaaS introduces prompt, retrieval, model, tool, and output boundaries. Tenant context must constrain retrieval and tool execution. Use approved models, data-loss prevention, content controls, grounding evaluation, prompt-injection defenses, tool allowlists, and human approval for high-impact actions.

Measure token or inference cost, latency, quality, safety, and tenant-level limits. Treat model versions and prompts as deployable artifacts with evaluation evidence and rollback. Never assume a managed AI service automatically supplies application-level authorization.

SaaS principle: tenant context is a security boundary, an operational dimension, a cost dimension, and a recovery dimension - not merely a database column.

Production Design Checklist

- Tenant context originates from trusted identity.
- Isolation tiers and cell placement are explicit.

- Data queries, caches, queues, and logs preserve tenant boundaries.
- Onboarding and offboarding are idempotent and auditable.
- AI retrieval and tool execution are constrained by tenant authorization.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Cross-tenant data exposure	Tenant filter is missing in one path	Contain affected cell, revoke access, audit all data paths	Automated tests prove isolation across APIs, caches, jobs, and analytics
Noisy neighbor affects latency	Shared quotas and hot partitions	Apply tenant quotas, rebalance cells, redesign partition keys	Per-tenant SLOs and capacity signals remain healthy under stress
Onboarding stalls halfway	Workflow is not idempotent or compensating	Resume from durable state or execute compensation	Repeated onboarding produces one valid tenant state
AI assistant reveals unrelated context	Retrieval boundary or tool authorization failed	Disable capability, investigate logs, rotate affected indexes	Evaluation suite proves tenant-scoped retrieval and tool access

Hands-on Lab 57: Design a Cell-Based SaaS Platform

Prerequisites

- A fictional multi-tenant product with three tenant tiers, two jurisdictions, and one AI-assisted feature.

Implementation

1. Define tenant tiers, regulatory constraints, expected scale, and unit economics.
2. Choose shared, pooled, cell-based, and dedicated isolation boundaries for identity, compute, data, keys, and logs.
3. Design tenant placement, routing, and a control-plane record with idempotent lifecycle states.
4. Define per-tenant quotas, SLOs, metering events, support access, export, and deletion procedures.
5. Create a failure scenario that removes one cell and prove unaffected cells continue serving.
6. Design AI retrieval and tool-execution guardrails with evaluation tests.
7. Produce an ADR comparing the selected cell design with a single shared deployment.

Validation Evidence

- Every request has trusted tenant context.
- Failure of one cell has bounded impact.
- Metering reconciles after duplicate events.
- AI tests reject cross-tenant retrieval.

Cleanup

- Delete synthetic tenant data and temporary service identities.
- Retain only sanitized diagrams, ADRs, and test evidence.

Interview Questions and Model Answers

1. Why use cells?

To limit blast radius, scale independently, and enable safer rollout and recovery.

2. What is the control-plane risk?

If tenant lifecycle or placement depends on it synchronously, a control-plane outage can become a data-plane outage.

3. How do you prevent noisy neighbors?

Use quotas, partition design, workload isolation, capacity signals, and tenant-aware SLOs.

4. What changes when adding AI?

Retrieval, prompt, model, tool, output, evaluation, privacy, and cost boundaries become explicit architecture concerns.

Chapter Summary

- Choose isolation from risk and economics.
- Carry tenant context through every control and signal.
- Use cells and rings to reduce correlated failure.
- Treat AI features as governed production systems.

Chapter 58 - Hybrid Cloud, Distributed Cloud, and Migration Factories

Assessing estates, building foundations, and moving workloads through repeatable migration waves

Learning Objectives

- Rationalize workloads using retire, retain, rehost, replatform, refactor, repurchase, and relocate decisions.
- Use discovery evidence and dependency mapping to create migration waves.
- Design hybrid identity, networking, operations, data, and security boundaries.
- Build a migration factory with repeatable intake, landing, validation, cutover, rollback, and optimization.

58.1 Strategy and Workload Rationalization

Migration is a portfolio transformation, not a sequence of copy operations. Define business drivers, deadlines, exit costs, regulatory obligations, modernization appetite, operating-model readiness, and success measures before selecting tooling.

Rationalization decisions must reflect technical fit and business value. A low-value unsupported application may be retired; a stable vendor product may be repurchased as SaaS; a tightly coupled estate may first be rehosted to meet a data-center exit and modernized later.

58.2 Discovery, Dependencies, and Wave Planning

Use inventory, utilization, application ownership, network flows, authentication, DNS, data movement, batch schedules, licenses, certificates, and support dependencies. Dependency confidence matters: unknown dependencies should increase wave risk and require additional observation.

Create waves around dependency groups and business calendars. Each wave requires entry criteria, foundation readiness, data plan, test plan, cutover sequence, rollback trigger, communications plan, and hypercare ownership.

58.3 Hybrid and Distributed Operations

Hybrid architecture spans identity federation, Cloud Interconnect or HA VPN, routing, DNS, certificate trust, logging, monitoring, patching, backup, configuration, and incident escalation. Treat the connection and management plane as products with SLOs and tested failover.

Google Distributed Cloud or other edge and hybrid platforms can extend consistent operations where latency, data sovereignty, disconnected operation, or local processing requires it. The operating model must clearly separate Google-managed, customer-managed, and shared responsibilities.

58.4 Migration Factory Operating Model

A migration factory standardizes intake, assessment, landing-zone readiness, application remediation, data synchronization, rehearsal, cutover, validation, and optimization. Standardization should reduce avoidable variance while preserving an exception path for unusual workloads.

Measure lead time, wave success, rollback frequency, defect escape, dependency surprises, post-cutover incidents, cost variance, and optimization backlog. A migration is not complete until operations, security, backup, cost ownership, and support have accepted the workload.

Migration principle: move evidence, ownership, and recovery capability with the workload - not only compute and data.

Production Design Checklist

- Business driver and exit criteria are explicit.

- Dependencies and ownership confidence are scored.
- Foundation, connectivity, IAM, logging, backup, and billing are ready.
- Cutover and rollback are rehearsed.
- Post-migration optimization and decommissioning have owners.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Wave misses hidden dependency	Discovery lacked runtime flows or owner validation	Pause cutover, restore service path, update dependency map	Rehearsal and observation cover critical dependency windows
Hybrid latency breaks application	Chatty dependency crosses WAN	Co-locate components, cache, batch, or redesign interaction	Latency test represents production traffic and failure modes
Cutover succeeds but operations fail	Runbooks, alerts, backup, or access were not accepted	Enter controlled hypercare and close operational gaps	Operations signs evidence before migration closure
Costs exceed business case	Sizing and licensing assumptions were wrong	Rightsize, schedule, modernize, renegotiate, or redesign	Actual unit cost is reconciled with forecast

Hands-on Lab 58: Build a Migration Wave Plan

Prerequisites

- An inventory of at least twelve fictional applications with owners, dependencies, utilization, data classification, and business deadlines.

Implementation

1. Define migration drivers, success measures, constraints, and non-negotiable business dates.
2. Score each application for business value, technical complexity, dependency confidence, and modernization opportunity.
3. Select a rationalization path and document the evidence for each choice.
4. Group applications into waves based on dependencies, risk, and learning value.
5. Create foundation, test, data, cutover, rollback, communications, and hypercare checklists for the first wave.
6. Run a tabletop rehearsal that introduces a hidden dependency and a failed data-validation check.
7. Revise the wave plan, metrics, and exception process based on the rehearsal.

Validation Evidence

- Every workload has an owner and rationalization decision.
- Wave dependencies are visible.
- Rollback has time and data boundaries.
- Operational acceptance is part of completion.

Cleanup

- Remove synthetic inventories containing sensitive-looking data.
- Store only sanitized migration artifacts and lessons learned.

Interview Questions and Model Answers

1. Why not migrate the easiest applications first?

Easy waves can create early learning, but dependency order, business deadlines, and foundation readiness may be more important.

2. What is wave entry criteria?

Evidence that foundation, application, data, testing, ownership, and rollback prerequisites are satisfied.

3. When is rehost appropriate?

When speed, compatibility, or deadline dominates and the future modernization plan is explicit.

4. What makes a migration factory effective?

Repeatable controls, measurable flow, clear exceptions, and feedback from every wave.

Chapter Summary

- Migration is a portfolio and operating-model change.
- Build waves from dependency evidence.
- Treat hybrid foundations as products.
- Close migration only after operational acceptance.

Chapter 59 - Application Modernization and Monolith Decomposition

Moving from tightly coupled systems to independently deployable, observable, resilient capabilities

Learning Objectives

- Select modernization approaches based on business value, coupling, risk, and team capability.
- Use decomposition patterns such as strangler, branch by abstraction, event interception, and modular monolith.
- Manage data ownership, distributed transactions, compatibility, and migration sequencing.
- Choose GKE, Cloud Run, managed services, or retained VMs based on workload characteristics.

59.1 Modernization Decision Framework

Modernization is justified when it changes business velocity, reliability, security, scalability, cost, or maintainability enough to repay transformation risk. Avoid decomposing merely to adopt fashionable products.

Assess domain boundaries, release coupling, change failure rate, testability, operational maturity, data ownership, latency sensitivity, compliance, and team structure. A modular monolith can be a better intermediate state than prematurely distributed microservices.

59.2 Decomposition Patterns

The strangler pattern routes selected capabilities to new implementations while the monolith continues serving the remainder. Branch by abstraction changes internal interfaces before moving implementations. Event interception captures useful domain events without rewriting the entire transaction path.

Use anti-corruption layers when new domain models must interact with legacy data or protocols. Define retirement criteria for old paths; otherwise the organization accumulates permanent dual systems.

59.3 Data Ownership and Consistency

Data decomposition is usually harder than compute decomposition. Establish authoritative ownership, migration states, reconciliation, idempotency, schema compatibility, and fallback. Avoid shared-database coupling that allows every service to update every table.

Use sagas, outbox patterns, change data capture, and compensating actions where atomic cross-service transactions are not possible. Explicitly choose consistency and failure behavior instead of inheriting it accidentally.

59.4 Platform and Operating Model

Cloud Run is appropriate for many stateless HTTP, event, job, and worker workloads; GKE provides deeper Kubernetes control and a broad platform surface; Compute Engine remains valid for specialized or legacy requirements. Managed databases, Pub/Sub, Eventarc, Workflows, and API products can remove undifferentiated operations.

Modernization also requires service ownership, on-call readiness, SLOs, dependency maps, secure delivery, telemetry, cost allocation, and incident response. More services without stronger operations create more failure modes, not more agility.

Modernization principle: decompose around business capability and ownership, not around technical layers or arbitrary repository boundaries.

Production Design Checklist

- Business value and exit criteria are measurable.

- Domain and data ownership are explicit.
- Compatibility and rollback exist during coexistence.
- Each new service has SLO, owner, runbook, and cost allocation.
- Legacy paths have retirement dates.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
New service and monolith diverge	Dual writes lack ordering or reconciliation	Choose source of truth, stop unsafe writes, reconcile data	Replay and reconciliation tests prove convergence
Latency increases after decomposition	Chatty synchronous calls replaced local calls	Batch, cache, redesign API, or use events	Performance budget passes at realistic dependency latency
Incidents cross many teams	Ownership and service boundaries are unclear	Create service catalog, escalation, SLO, and dependency map	Incident commander can identify accountable responders quickly
Legacy never retires	No measurable extraction completion criteria	Define traffic, data, and feature exit milestones	Old path receives no production traffic and can be removed

Hands-on Lab 59: Extract a Capability from a Monolith

Prerequisites

- A sample monolith with one independently valuable capability, synthetic data, and a repeatable test suite.

Implementation

1. Map the capability, data, callers, transactions, dependencies, and failure behavior.
2. Choose a strangler or branch-by-abstraction approach and record the decision.
3. Define the new API or event contract with compatibility and idempotency rules.
4. Implement the capability on Cloud Run or GKE with separate telemetry and ownership.
5. Migrate reads, then writes, using reconciliation and controlled traffic shifting.
6. Inject dependency failure and prove fallback or compensation.
7. Remove the old path after exit criteria are met and publish the modernization evidence.

Validation Evidence

- Behavior remains compatible.
- Data reconciliation reaches zero unexplained variance.
- New service has independent deployment and rollback.
- Old code path is removable.

Cleanup

- Delete temporary services, images, topics, and synthetic data.
- Retain sanitized architecture, contract, test, and migration evidence.

Interview Questions and Model Answers

1. What is the strangler pattern?

A progressive replacement approach that routes selected capabilities to new implementations while legacy functions continue temporarily.

2. Why is shared database access risky?

It preserves coupling, bypasses ownership, and makes independent change and recovery difficult.

3. When should a monolith remain?

When cohesion is high, scale is manageable, team boundaries do not require separation, and operational simplicity has greater value.

4. What proves modernization success?

Improved business and engineering outcomes plus retired legacy paths, not merely more services.

Chapter Summary

- Modernize for measurable value.
- Decompose domain, data, and ownership together.
- Plan coexistence, reconciliation, and retirement.
- Strengthen operations as service count grows.

Chapter 60 - Multi-Region and Global Application Design

Designing global ingress, data, dependency, failover, and recovery behavior

Learning Objectives

- Compare active-passive, active-active, and cell-based regional topologies.
- Design global ingress and dependency locality with Google Cloud load balancing and DNS.
- Choose data replication and consistency models aligned to RTO, RPO, and user experience.
- Create deterministic failover, fallback, and recovery orchestration.

60.1 Global Topologies

Active-passive topologies concentrate writes in one region and maintain a warm or cold recovery region. They can simplify consistency but require tested activation. Active-active topologies serve multiple regions simultaneously and can improve latency and availability, but introduce routing, data, conflict, and operational complexity.

Cell-based topologies bound tenant or customer groups to independent regional cells. Global systems often combine patterns: global stateless ingress, regional cells, and a separately governed global control plane.

60.2 Ingress and Dependency Locality

Global external load balancing can route users to healthy backends, but application availability still depends on data, identity, secrets, queues, third-party APIs, and management paths. Build dependency maps by region and classify each dependency as local, replicated, global, or external.

Design graceful degradation for noncritical features. A regional outage should not become a global outage because recommendation, analytics, or administrative functionality is unavailable.

60.3 Data Consistency and Recovery

Select data products and topology from consistency, latency, availability, sovereignty, and operational requirements. Spanner can support strongly consistent multi-region relational designs; other databases may use replicas, backups, export, or application-level conflict handling.

RPO is not only a backup setting. It depends on replication lag, transaction semantics, queue retention, object versioning, and reconciliation. Failover without validated data state can restore traffic to an inconsistent system.

60.4 Orchestration, Fallback, and Testing

Automate health detection cautiously and separate detection from business authorization where false failover is costly. Define traffic drain, write fencing, data validation, activation, smoke testing, communications, and rollback. Fallback is a distinct operation with its own consistency and capacity risks.

Rehearse regional loss, dependency loss, and partial control-plane impairment. Track time to detect, decide, execute, validate, communicate, and stabilize.

Global-design principle: a second region is not a recovery strategy until traffic, data, dependencies, people, and runbooks have been tested together.

Production Design Checklist

- Topology matches business RTO/RPO.
- Every critical dependency has regional behavior.
- Write ownership and fencing are explicit.
- Failover and fallback are rehearsed.
- Graceful degradation preserves core user journeys.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Traffic moves but application fails	Data or identity dependency is unavailable in target region	Stop or limit traffic, restore dependency, validate end-to-end	Synthetic journey succeeds from target region
Split brain or duplicate writes	Write fencing and authority are unclear	Fence old writer, reconcile, establish one authority	No concurrent unauthorized write paths remain
Automatic failover flaps	Health signal is noisy or too local	Use multi-signal confirmation and hold-down periods	Failover triggers only on tested conditions
Failback causes overload	Capacity and backlog were not modeled	Scale target, drain backlog, shift gradually	Error budget and latency remain healthy during failback

Hands-on Lab 60: Run a Regional Failover Exercise

Prerequisites

- A low-cost two-region sample application with synthetic data, observable health, and a documented RTO/RPO.

Implementation

- Document serving, data, identity, secret, queue, DNS, and external dependencies by region.
- Define failure detection, decision authority, write fencing, traffic shift, validation, and failback.
- Create a baseline using synthetic transactions, latency, error rate, data checks, and queue depth.
- Simulate primary-region unavailability without destroying evidence.
- Execute failover, record each timeline milestone, and validate data state and critical journeys.
- Recover the original region, reconcile state, and perform controlled failback.
- Publish the observed RTO/RPO, deviations, and remediation backlog.

Validation Evidence

- Target-region capacity is adequate.
- No split-brain writes occur.
- Critical journeys meet recovery objective.
- Failback is completed without unexplained data variance.

Cleanup

- Restore normal traffic and remove temporary test capacity.
- Archive sanitized timelines, dashboards, and recovery evidence.

Interview Questions and Model Answers

- Why is multi-region not automatically active-active?

Serving and data-write patterns, consistency, dependencies, and business routing may still be active-passive.

- What is write fencing?

A mechanism that prevents an old or unauthorized writer from accepting mutations during failover.

- Why test failback?

Returning to the preferred topology can introduce its own overload, data, and routing risks.

4. What should RTO measurement include?

Detection, decision, execution, validation, communication, and stabilization - not only infrastructure activation.

Chapter Summary

- Choose topology from business objectives.
- Map every regional dependency.
- Protect data authority during failover.
- Test failover and failback as end-to-end operations.

Chapter 61 - Security Incident Response and Cloud Forensics

Preparing, containing, investigating, recovering, and learning from Google Cloud security incidents

Learning Objectives

- Establish cloud-specific security incident roles, severity, communications, and decision authority.
- Collect and preserve evidence from audit logs, workload telemetry, identity, network, data, and security products.
- Contain compromised identities and workloads without destroying evidence unnecessarily.
- Recover through trusted rebuild, validation, credential rotation, and control improvement.

61.1 Incident Lifecycle and Command

Security response requires preparation, identification, triage, containment, eradication, recovery, and learning. Define incident commander, security lead, operations lead, forensics lead, communications, legal, privacy, and business roles before an event.

Use severity criteria based on affected data, privilege, persistence, blast radius, business impact, regulatory obligation, and attacker capability. Maintain a secure out-of-band coordination channel and an evidence log.

61.2 Evidence Sources

Cloud Audit Logs, IAM policy history, service-account activity, Security Command Center findings, Google Security Operations detections, VPC Flow Logs, firewall logs, load-balancer logs, DNS logs, Cloud Logging, GKE audit logs, application logs, and data-access events can establish timeline and scope.

Preserve original logs and metadata with controlled access, retention, time synchronization, and chain-of-custody records. Export evidence before changing retention or deleting compromised resources.

61.3 Containment and Forensics

Containment can include disabling principals, revoking sessions, removing IAM bindings, rotating secrets or keys, restricting egress, isolating projects, blocking indicators, or scaling a compromised workload to zero. Choose actions that reduce harm while preserving enough evidence to understand persistence and blast radius.

Avoid logging into a compromised instance and manually “cleaning” it. Snapshot disks where appropriate, preserve metadata, and rebuild from trusted images and source. Treat attacker-created identities, keys, scheduled jobs, images, and CI/CD paths as persistence mechanisms.

61.4 Recovery and Learning

Recovery requires a trusted control plane, clean deployment artifacts, validated data, rotated credentials, restored monitoring, and heightened detection. Re-enable capabilities progressively and define clear rollback triggers.

Post-incident review should identify technical and organizational contributors without blame. Convert findings into preventive, detective, and recovery controls with owners, due dates, and verification.

Forensics principle: containment that destroys the only useful evidence may shorten the incident but lengthen uncertainty, regulatory exposure, and recurrence.

Production Design Checklist

- Roles, severity, communications, and legal paths are pre-defined.
- Critical logs are centralized and protected.
- Containment playbooks exist for identity, workload, network, and data incidents.
- Trusted rebuild and credential rotation are rehearsed.
- Lessons become verified control improvements.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Audit trail is incomplete	Data access or workload logs were not enabled or retained	Preserve available evidence and document gaps	Logging baseline covers required services and retention
Attacker persists after rotation	Additional keys, identities, CI/CD or scheduled jobs remain	Hunt for persistence and rebuild trusted paths	No unexplained credentials or execution paths remain
Containment causes outage	Blast radius and business dependencies were unknown	Use phased containment and business prioritization	Playbooks include service-impact choices and approvals
Evidence integrity is questioned	Chain of custody and access controls are missing	Hash, restrict, timestamp, and document evidence handling	Independent reviewer can verify evidence history

Hands-on Lab 61: Run a Compromised Workload Identity Simulation

Prerequisites

- A sandbox project, synthetic workload identity, centralized logging, and a pre-approved simulation plan.

Implementation

1. Create incident roles, severity, communication channel, evidence log, and stop conditions.
2. Generate controlled suspicious activity using a test identity with limited permissions.
3. Detect the activity through audit logs or a security rule and open an incident timeline.
4. Preserve identity, policy, network, workload, and application evidence before containment.
5. Disable or constrain the identity, rotate affected secrets, and isolate the workload.
6. Rebuild from trusted configuration, validate user journeys, and monitor for recurrence.
7. Conduct a retrospective and convert findings into one preventive, one detective, and one recovery improvement.

Validation Evidence

- Detection identifies the test activity.
- Evidence predates containment changes.
- Compromised access is removed.
- Recovery uses trusted artifacts and produces a complete timeline.

Cleanup

- Delete synthetic identities and test findings after preserving sanitized evidence.
- Restore only approved baseline policies.

Interview Questions and Model Answers

1. Why centralize audit logs?

To reduce tampering risk, preserve evidence beyond project compromise, and support cross-project investigation.

2. What is trusted rebuild?

Recreating the workload from verified source, images, configuration, identities, and secrets rather than cleaning the compromised runtime.

3. Why can credential rotation be insufficient?

Attackers may have created alternate identities, keys, workloads, pipelines, or scheduled persistence.

4. What is chain of custody?

A documented record of evidence collection, integrity, access, transfer, and storage.

Chapter Summary

- Prepare roles and evidence paths before incidents.
- Contain with awareness of evidence and business impact.
- Rebuild from trusted sources.
- Turn lessons into verified controls.

Chapter 62 - Resilience Engineering, Game Days, and Chaos Experiments

Verifying that systems preserve critical user outcomes during failure

Learning Objectives

- Define resilience from user journeys, SLOs, RTO, RPO, and failure assumptions.
- Design safe experiments with hypotheses, blast radius, abort conditions, and observability.
- Test compute, network, dependency, quota, data, and control-plane failure modes.
- Run game days that improve technical systems and organizational coordination.

62.1 Resilience as a Verified Property

Resilience is the demonstrated ability to continue or recover critical outcomes under expected failure. Redundancy alone is not proof: replicas may share a quota, region, identity, configuration error, or dependency.

Begin with critical user journeys and error budgets. Identify failure domains, recovery mechanisms, manual dependencies, and hidden assumptions. Test the mechanism that should protect the user, not merely the component that fails.

62.2 Experiment Design

A good chaos experiment has a steady-state definition, hypothesis, injection, scope, observation window, abort criteria, owner, communication plan, and recovery procedure. Start with tabletop and nonproduction, then increase realism as evidence and confidence improve.

Examples include draining a backend, denying a dependency, introducing latency, exhausting a quota, stopping a worker, revoking a permission, corrupting a message, or removing a region from service. Avoid uncontrolled destructive testing.

62.3 Tooling and Automation

Google Cloud does not require one universal chaos product. Use load-balancer controls, instance-group actions, GKE disruption mechanisms, network controls, fault-injection tools, test doubles, Terraform, scripts, and approved third-party platforms. Keep experiment code versioned and reviewed.

Telemetry must show user impact, dependency behavior, autoscaling, retry amplification, queue depth, data correctness, and recovery time. A successful experiment can reveal a defect; success means learning safely, not proving the system never fails.

62.4 Game Days and Readiness

Game days combine technology, people, process, and communication. Include incident command, operations, application, security, data, networking, support, and business observers. Use injects that require decisions, not only commands.

End with a remediation backlog, updated runbooks, changed alerts, clarified ownership, and a retest date. Track time to detect, assemble, decide, mitigate, communicate, and recover.

Resilience principle: the experiment is safe when blast radius and recovery are controlled; the system is resilient when critical outcomes remain within agreed objectives.

Production Design Checklist

- Critical journeys and steady state are measurable.
- Hypothesis, scope, abort, and rollback are documented.
- Observability covers user and dependency effects.

- Incident roles participate in game days.
- Findings are retested after remediation.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Experiment causes uncontrolled impact	Abort path or scope was not enforced	Stop injection, restore baseline, invoke incident process	Future test has automated safety guard and explicit owner
No useful result	Steady state and hypothesis were vague	Redefine measurable outcome and repeat at smaller scope	Telemetry proves or disproves one hypothesis
Retries amplify outage	Backoff, jitter, limits, or idempotency are missing	Throttle, disable retry storm, drain backlog safely	Failure test shows bounded retries and stable recovery
Runbook fails during game day	Steps assume unavailable tools or access	Create alternate access and validate prerequisites	Different operator can execute the revised runbook

Hands-on Lab 62: Conduct a Controlled Resilience Game Day

Prerequisites

- A nonproduction service with SLOs, synthetic traffic, dependency telemetry, and an approved failure scenario.

Implementation

1. Choose one critical user journey and define its steady state and error-budget limits.
2. Write a hypothesis for one dependency or regional failure.
3. Define blast radius, abort thresholds, observers, incident roles, and recovery steps.
4. Capture baseline traffic, latency, errors, saturation, queue depth, and data checks.
5. Inject the failure and run the incident process without skipping communication.
6. Recover, compare observed behavior with hypothesis, and identify hidden coupling.
7. Assign remediation owners and schedule a retest.

Validation Evidence

- Safety controls work.
- User impact is measurable.
- Incident roles and communication execute.
- Remediation items have owners and retest dates.

Cleanup

- Remove injection rules, temporary capacity, and test identities.
- Verify all metrics and configurations return to baseline.

Interview Questions and Model Answers

1. What is steady state?

A measurable description of normal user and system behavior used to evaluate the experiment.

2. Why inject dependency latency?

Latency often triggers retries, thread exhaustion, queue growth, and cascading failure before outright dependency loss.

3. What makes a game day different from a technical test?

It exercises people, decisions, communications, and cross-team coordination as well as technology.

4. What is a successful chaos experiment?

One that safely produces trustworthy learning and leads to verified improvement.

Chapter Summary

- Define resilience from user outcomes.
- Use hypotheses and safety controls.
- Observe cascading behavior, not only failed components.
- Retest after remediation.

Chapter 63 - Performance Engineering and Capacity Planning

Building performance budgets, representative tests, bottleneck evidence, and safe scaling plans

Learning Objectives

- Create workload models from user behavior, growth, seasonality, and dependency limits.
- Use performance budgets and test types appropriate to risk.
- Diagnose CPU, memory, storage, network, database, queue, and application bottlenecks.
- Convert test evidence into capacity, autoscaling, quota, and cost plans.

63.1 Workload Model and Performance Budget

Performance engineering begins with traffic shape: request mix, payload size, concurrency, geographic distribution, read/write ratio, cache hit rate, batch windows, queue behavior, and growth. Average load hides peaks and correlations.

Define budgets for end-to-end latency, server time, dependency time, queue delay, error rate, throughput, saturation, and cost per transaction. Include percentile targets rather than only averages.

63.2 Test Types and Environments

Use smoke, baseline, load, stress, spike, soak, failover, scalability, and capacity tests for different questions. A load test proves expected demand; a stress test discovers failure boundaries; a soak test finds leaks and slow degradation; a failover test measures recovery under load.

Representative tests require realistic data distribution, authentication, caching, network path, dependency behavior, and autoscaling settings. Protect shared environments and define stop conditions.

63.3 Bottleneck Analysis

Correlate user latency with application traces, CPU throttling, memory pressure, garbage collection, connection pools, storage latency, database locks, query plans, cache behavior, queue age, network loss, and downstream saturation.

Avoid tuning one metric in isolation. Increasing concurrency can improve throughput while causing database contention. More retries can hide transient errors while exhausting resources.

63.4 Capacity, Autoscaling, and Quotas

Capacity plans combine baseline, peak, failure reserve, growth, deployment surge, and recovery backlog. Autoscaling needs a meaningful signal, target, minimum, maximum, stabilization behavior, and dependency awareness.

Request quotas and reservations before the event, not during it. Create rollback and load-shedding mechanisms for demand above safe capacity.

Performance principle: capacity is an end-to-end property constrained by the narrowest dependency, not by the largest compute tier.

Production Design Checklist

- Workload model represents peak and failure conditions.
- Budgets include percentiles and cost.
- Test data and dependency behavior are realistic.
- Autoscaling protects downstream dependencies.
- Quotas and recovery backlog are included in capacity.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Test passes but production is slow	Test mix, geography, data, or dependencies are unrealistic	Rebuild workload model from production evidence	Test and production profiles align within agreed variance
Autoscaling increases errors	Scaling signal is late or dependency is saturated	Use earlier signal, cap concurrency, protect dependency	Scale event improves SLO without downstream collapse
P99 latency grows over time	Leak, fragmentation, cache churn, or backlog	Run soak analysis and correlate traces with resource trends	Long-duration test remains within budget
Failover region overloads	Failure reserve and backlog were omitted	Pre-scale, load shed, prioritize, or redesign recovery	Failover capacity supports critical journeys

Hands-on Lab 63: Create a Performance Qualification Report

Prerequisites

- A sample service, realistic synthetic data, telemetry, and a defined business traffic forecast.

Implementation

1. Build a workload model with request mix, concurrency, geography, data size, and growth.
2. Define latency, throughput, error, saturation, recovery, and cost budgets.
3. Run smoke and baseline tests to validate the environment.
4. Run load, spike, soak, and dependency-latency tests with stop conditions.
5. Identify the first bottleneck using traces, metrics, logs, and database evidence.
6. Tune one hypothesis at a time and repeat the same test.
7. Publish qualified capacity, autoscaling limits, quota requirements, and residual risk.

Validation Evidence

- Test traffic matches the model.
- P95 and P99 are reported.
- Bottleneck claims have correlated evidence.
- Capacity includes failure reserve and cost.

Cleanup

- Stop load generators and remove temporary large resources.
- Delete synthetic data not required for the report.

Interview Questions and Model Answers

1. Why are averages insufficient?

They hide tail latency and the experience of the slowest significant user population.

2. What is a soak test?

A long-duration test used to expose leaks, backlog, cache churn, and gradual degradation.

3. Why can autoscaling worsen failure?

It can amplify load against a saturated database, queue, API, or quota.

4. What is performance qualification?

Evidence that a specific version and architecture meet defined budgets under representative conditions.

Chapter Summary

- Model real traffic and failure conditions.
- Use the right test for the question.
- Correlate end-to-end evidence.
- Plan capacity, quotas, reserve, and cost together.

Chapter 64 - FinOps and Enterprise Cost Optimization

Connecting cloud spend to ownership, unit economics, commitments, and engineering decisions

Learning Objectives

- Build a FinOps operating model across engineering, finance, procurement, product, and leadership.
- Allocate cost through hierarchy, labels, billing export, and unit metrics.
- Evaluate rightsizing, scheduling, architecture changes, and commitments using evidence.
- Create guardrails that prevent waste without blocking delivery.

64.1 FinOps Operating Model

FinOps is a continuous collaboration model, not a quarterly cost-cutting exercise. Establish ownership, reporting cadence, anomaly response, forecasting, optimization backlog, commitment governance, and exception handling.

Engineers need timely, contextual signals; finance needs forecast and allocation; product leaders need unit economics and business value. A shared cost language reduces conflict.

64.2 Allocation and Unit Economics

Use organization, folder, project, billing account, labels, tags, and service metadata to allocate spend. Export detailed billing data to BigQuery for trend, anomaly, reservation, and unit-cost analysis.

Unit metrics such as cost per tenant, transaction, build, gigabyte processed, model request, or active user connect architecture decisions to product value. Unallocated shared cost should be visible and governed rather than hidden.

64.3 Optimization and Commitments

Rightsize compute, storage, database, logs, network egress, and data processing based on utilization and service objectives. Schedule nonproduction resources, remove orphaned assets, tune retention, and redesign expensive data movement or query patterns.

Committed use discounts and reservations can reduce unit cost when demand is predictable. Model baseline, growth, seasonality, failure reserve, migration, and downside risk. Do not cover speculative usage with irreversible commitment.

64.4 Guardrails and Engineering Feedback

Budgets and alerts notify; quotas, policy, automation, and approval controls enforce. Use progressive guardrails: visibility, recommendations, automated cleanup of clearly temporary resources, and stronger controls for high-risk spend.

Optimization must preserve reliability, security, performance, and delivery capability. Record accepted cost for resilience and compliance instead of labeling all redundancy as waste.

FinOps principle: optimize business value and unit economics, not the smallest possible invoice.

Production Design Checklist

- Spend is allocated to owner and product.
- Unit-cost metrics are defined.
- Forecast includes growth and failure reserve.
- Commitments use conservative evidence.
- Optimization changes are validated against SLO and risk.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Cost is unallocated	Hierarchy and labels do not map to ownership	Create allocation standard and shared-cost rule	Unallocated spend falls below agreed threshold
Commitment is underused	Demand forecast was optimistic or workload migrated	Reassign eligible usage, adjust future purchases	Coverage and utilization meet governance target
Logging cost spikes	Verbose logs, high-cardinality fields, or long retention	Filter, sample, route, and tune retention safely	Required evidence remains while ingest and storage stabilize
Optimization causes incident	Reliability reserve or dependency capacity was removed	Restore safe capacity and review decision controls	SLO remains healthy after revised optimization

Hands-on Lab 64: Build a FinOps Optimization Plan

Prerequisites

- A sanitized billing export or synthetic cost dataset with project, service, SKU, labels, and usage data.

Implementation

1. Map projects and shared services to owners, products, environments, and cost centers.
2. Create a BigQuery analysis for monthly trend, anomaly, top services, unallocated cost, and unit metric.
3. Identify ten opportunities across rightsizing, idle resources, retention, data processing, egress, and architecture.
4. Classify each opportunity by savings, engineering effort, risk, reversibility, and SLO impact.
5. Model conservative commitment coverage using baseline and downside scenarios.
6. Define budgets, anomaly ownership, cleanup automation, and exception workflow.
7. Publish a ninety-day optimization backlog with validation and realized-savings evidence.

Validation Evidence

- Allocation rules are reproducible.
- Savings estimates include assumptions.
- SLO and risk are considered.
- Realized savings are measured after implementation.

Cleanup

- Remove temporary billing datasets and queries containing sensitive allocation data.
- Retain only sanitized analysis and approved reports.

Interview Questions and Model Answers

1. What is unit economics?

The cost to deliver a measurable product outcome such as a transaction, tenant, user, or data unit.

2. Why can high commitment coverage be risky?

It can create underutilized obligations when demand, architecture, or migration changes.

3. Do budgets stop spend?

Budgets primarily provide tracking and alerts; enforcement requires quotas, policy, automation, or approval controls.

4. How should reliability cost be treated?

As an explicit business decision tied to RTO, RPO, and service objectives, not automatically as waste.

Chapter Summary

- FinOps is shared operational practice.
- Allocate spend and measure unit cost.
- Use conservative commitment evidence.
- Protect reliability while optimizing.

Chapter 65 - Platform Engineering and Internal Developer Platforms

Building secure, self-service paved roads that improve developer flow and production outcomes

Learning Objectives

- Treat the platform as a product with users, outcomes, roadmap, and support.
- Design golden paths for common workload classes using Terraform, CI/CD, and policy.
- Provide self-service without bypassing security, cost, and operational ownership.
- Measure adoption, developer experience, delivery performance, reliability, and platform toil.

65.1 Platform as a Product

An internal developer platform is a set of capabilities, interfaces, and operating practices that help teams create and operate software safely. It is not merely a portal or a Kubernetes cluster.

Identify user personas, journeys, top friction, service classes, adoption barriers, and success measures. Build a roadmap from developer and operator evidence rather than from platform-team preferences.

65.2 Paved Roads and Golden Paths

A golden path combines approved architecture, reusable modules, source templates, build and deployment pipelines, identity, secrets, telemetry, SLO defaults, cost allocation, runbooks, and recovery. It should be the easiest safe path while preserving an exception process.

Provide separate paths for Cloud Run services, GKE workloads, event consumers, data pipelines, and scheduled jobs when their operational needs differ. Version templates and publish compatibility and deprecation policy.

65.3 Self-Service and Governance

Self-service requests should validate ownership, environment, data class, region, budget, support model, and service objective. Automation provisions projects, APIs, IAM, network attachment, pipelines, secrets, telemetry, and policy evidence.

Use policy as code, organization policy, IAM conditions or deny rules, VPC Service Controls, Binary Authorization, budgets, and automated tests. Exceptions must be time-bound, approved, visible, and reviewed.

65.4 Platform Metrics and Reliability

Measure time to first deployment, change lead time, deployment success, developer satisfaction, path adoption, exception rate, platform incidents, support volume, cost per environment, and percentage of services with SLOs and recovery evidence.

Operate the platform with its own SLOs, on-call, security model, capacity plan, release process, and disaster recovery. A broken platform can block the entire engineering organization.

Platform principle: standardization earns adoption when it removes toil and risk; mandates without useful paved roads create shadow platforms.

Production Design Checklist

- Platform users and journeys are explicit.
- Golden paths include operations and recovery, not only deployment.
- Exceptions are governed and time-bound.
- Platform itself has SLOs and ownership.
- Metrics cover flow, quality, reliability, and satisfaction.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Teams avoid platform	Path is slower or less capable than manual work	Research user friction and simplify the paved road	Adoption rises without coercive exception growth
Template updates break services	Versioning and compatibility policy are missing	Pin versions, test migrations, publish deprecation	Consumers upgrade through a documented path
Self-service creates privilege sprawl	Requests bypass identity and ownership controls	Add policy, approval, lifecycle, and access review	Every provisioned resource has owner and least privilege
Platform outage stops all delivery	No platform SLO or recovery architecture	Add redundancy, queueing, break-glass, and DR	Critical deployment and recovery paths are tested

Hands-on Lab 65: Design an Internal Developer Platform Product

Prerequisites

- Three developer personas and two workload classes with known delivery, security, and operational pain points.

Implementation

1. Interview personas and define the highest-value developer journeys and measurable outcomes.
2. Choose one Cloud Run path and one GKE or data-pipeline path.
3. Design the request contract: owner, environment, data class, region, SLO, budget, and support tier.
4. Create the component map for project creation, Terraform modules, CI/CD, identity, secrets, networking, telemetry, policy, and recovery.
5. Define versioning, exception, deprecation, support, and incident processes.
6. Create a platform scorecard for adoption, lead time, quality, reliability, cost, and satisfaction.
7. Run a tabletop onboarding and platform-outage scenario and revise the design.

Validation Evidence

- A developer can follow the path without privileged manual steps.
- Operational ownership is created automatically.
- Exceptions have expiry and review.
- Platform failure has a tested fallback.

Cleanup

- Delete any temporary prototype projects and service accounts.
- Retain sanitized product requirements and architecture evidence.

Interview Questions and Model Answers

1. What is a golden path?

A supported, reusable, secure, observable, and operable route for a common workload class.

2. Why is a portal not an IDP?

A portal is only an interface; the platform includes automation, policy, runtime, operations, support, and product management.

3. How do you avoid platform lock-in?

Use clear contracts, versioned modules, exportable configuration, and an exception path.

4. What should platform teams measure?

Developer flow, adoption, quality, reliability, security, cost, satisfaction, and platform toil.

Chapter Summary

- Treat the platform as a product.
- Make the safe path the easiest path.
- Govern self-service and exceptions.
- Operate the platform as critical production infrastructure.

Chapter 66 - Advanced Troubleshooting and Incident Command

Using evidence, layered hypotheses, and disciplined coordination during complex failures

Learning Objectives

- Apply an evidence-driven troubleshooting method under uncertainty.
- Investigate across client, edge, network, identity, compute, data, messaging, and external dependency layers.
- Separate incident command from technical investigation and business communication.
- Recover safely, preserve learning, and prevent recurrence.

66.1 Evidence-Driven Method

Begin with impact, scope, start time, affected journeys, recent changes, and reliable signals. Form hypotheses, predict observable evidence, run the safest discriminating test, and update confidence. Avoid random changes that erase causality.

Create a timeline with facts, hypotheses, decisions, actions, owners, and results. Mark uncertainty explicitly. Prefer reversible mitigations that reduce user impact while preserving evidence.

66.2 Layered Troubleshooting

Investigate from user experience inward: DNS and certificates, global load balancing and Cloud Armor, network routing and firewall, identity and permissions, compute health and autoscaling, application saturation, database and cache behavior, queues and event delivery, and external dependencies.

Use Cloud Monitoring, Cloud Logging, Trace, Profiler, Error Reporting, audit logs, Service Health, Network Intelligence Center, Security Command Center, and application-specific tools. Correlation does not prove causation; use controlled tests.

66.3 Incident Command

The incident commander owns coordination, priorities, roles, cadence, and decision log. Technical leads investigate; communications provides consistent updates; a scribe maintains timeline and action record. One person should not perform every role during a major incident.

Define severity, update interval, escalation, business decision authority, and stop conditions. Protect responders from interruption and avoid unreviewed parallel changes.

66.4 Recovery and Follow-up

Mitigation reduces impact; recovery returns to a stable trusted state; resolution closes the incident; problem management removes contributors. Keep these stages distinct.

After stabilization, validate data, backlog, scaling, security, and customer journeys. Conduct a blameless review, identify systemic contributors, assign actions, and verify them through tests or game days.

Troubleshooting principle: every change should answer a question, reduce impact, or restore a known-good state - otherwise it is noise.

Production Design Checklist

- Impact and scope are stated in user terms.
- Timeline distinguishes fact from hypothesis.
- Roles and communication cadence are explicit.
- Changes are coordinated and reversible.
- Follow-up actions have verification methods.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Many teams change systems at once	Incident command and change control are absent	Assign commander, freeze uncoordinated changes	Timeline shows one owner and result for every change
Metrics look healthy but users fail	Wrong SLI or missing journey telemetry	Use synthetic transaction, traces, and edge logs	User journey and component signals correlate
Mitigation creates data inconsistency	Recovery action ignored write authority or backlog	Fence writes, reconcile, restore controlled flow	Data checks and queue state are clean
Postmortem actions repeat	Actions are vague or unverified	Convert to specific control and test	Game day or automated check proves improvement

Hands-on Lab 66: Run an Advanced Incident Simulation

Prerequisites

- A sample multi-service application, dashboards, logs, traces, runbooks, and at least four participants.

Implementation

- Assign incident commander, operations lead, application lead, communications, and scribe.
- Inject a combined symptom such as elevated latency, queue growth, and one misleading healthy metric.
- State impact, scope, start time, recent changes, and first hypotheses.
- Build a layered investigation plan and run the safest discriminating tests.
- Select a reversible mitigation and record decision, owner, result, and rollback.
- Validate recovery across user journey, data, queue, scaling, and security.
- Write a post-incident review with systemic contributors and verified follow-up actions.

Validation Evidence

- Roles remain separate.
- Timeline is complete.
- Mitigation is linked to evidence.
- Recovery validates more than one component metric.

Cleanup

- Remove injected faults and reset test data.
- Archive only sanitized incident materials.

Interview Questions and Model Answers

- What is a discriminating test?

A safe test whose result meaningfully increases or decreases confidence in competing hypotheses.

- Why separate incident commander from technical lead?

Coordination and deep investigation require different attention and prevent one person becoming a bottleneck.

- What is mitigation versus resolution?

Mitigation reduces impact; resolution addresses the underlying incident sufficiently to close it.

- What makes a postmortem action strong?

A specific owner, deadline, systemic control, and verification method.

Chapter Summary

- Start with user impact and evidence.
- Investigate layer by layer.
- Use disciplined incident roles.
- Verify recovery and corrective actions.

Chapter 67 - Architecture Governance, Compliance, and Evidence

Translating obligations into testable controls and audit-ready evidence without blocking delivery

Learning Objectives

- Translate laws, standards, contracts, and policies into cloud control objectives.
- Design preventive, detective, and corrective controls across hierarchy, identity, network, data, build, runtime, and operations.
- Engineer durable evidence with ownership, integrity, retention, and reproducibility.
- Run architecture review boards and exception processes as risk-management systems.

67.1 From Obligation to Control Objective

Compliance begins with an obligation, not a product. Translate the obligation into a control objective, scope, accountable owner, implementation, evidence, frequency, and exception rule. Separate mandatory requirements from organization-specific policy choices.

Use data classification, system boundary, residency, retention, identity, segregation of duties, recovery, monitoring, and vendor obligations to define scope. A control that cannot produce evidence is difficult to govern.

67.2 Preventive, Detective, and Corrective Controls

Preventive controls include organization policy, IAM deny, VPC Service Controls, firewall policy, Binary Authorization, approved modules, and deployment policy. Detective controls include audit logs, Security Command Center, Asset Inventory, policy analysis, drift detection, and Google Security Operations. Corrective controls include workflow automation, access revocation, quarantine, rebuild, and ticketed remediation.

Layer controls because each can fail. Design ownership and recovery for the control plane itself.

67.3 Evidence Engineering

Evidence should be attributable, time-bound, scoped, tamper-resistant, retained, and reproducible. Examples include policy evaluation, deployment attestations, access reviews, restore results, SLO reports, incident tests, vulnerability remediation, and exception approvals.

Automate evidence collection where possible but keep interpretation and sampling. Store evidence separately from the workload when compromise of the workload could alter it.

67.4 Review Boards and Exceptions

Architecture review boards should focus on material risk, unusual decisions, cross-platform impact, and exceptions. Standard workloads should move through pre-approved patterns and automated checks.

Exceptions require business justification, compensating controls, owner, expiry, review date, and measurable remediation. Permanent undocumented exceptions are policy failure.

Governance principle: the strongest control is not the one with the most documentation; it is the one that is clear, testable, monitored, recoverable, and owned.

Production Design Checklist

- Every obligation maps to a control objective.
- Control scope and owner are explicit.
- Evidence is protected and reproducible.
- Standard patterns are automated.
- Exceptions have compensating controls and expiry.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Audit evidence is missing	Control implementation did not define evidence	Reconstruct carefully and redesign collection	Future evidence is automated and sampled
Policy blocks legitimate deployment	Control scope is too broad or context is missing	Use canary folder, conditions, exception workflow	Required workload passes with documented boundary
Exception never expires	No owner or review automation	Escalate, add expiry and remediation	Expired exceptions close or renew formally
Control passes but risk remains	Evidence tests configuration, not outcome	Add restore, attack, incident, or user-journey test	Operational evidence demonstrates objective

Hands-on Lab 67: Create an Audit-Ready Control Evidence Pack

Prerequisites

- A fictional regulated workload and five obligations covering identity, data, deployment, logging, and recovery.

Implementation

1. Translate each obligation into control objective, scope, owner, frequency, evidence, and exception rule.
2. Map preventive, detective, and corrective controls across Google Cloud services and team processes.
3. Create automated or scripted evidence for policy, IAM, deployment provenance, logs, and restore testing.
4. Store evidence in a restricted project or repository with retention and integrity controls.
5. Run an independent review that attempts to reproduce the evidence.
6. Create one time-bound exception with compensating controls and expiry.
7. Publish the control matrix, evidence index, gaps, and remediation plan.

Validation Evidence

- Evidence can be reproduced.
- Scope and timestamp are clear.
- Exception has owner and expiry.
- Operational outcome is tested for at least one control.

Cleanup

- Delete temporary sensitive evidence and revoke reviewer access.
- Retain only approved sanitized control artifacts.

Interview Questions and Model Answers

1. What is a control objective?

A clear statement of the risk-reducing outcome that a control must achieve.

2. Why separate evidence storage?

To reduce the chance that workload compromise or administrators can alter the evidence.

3. What is a compensating control?

An alternate safeguard that reduces risk when the standard control cannot be applied.

4. What should an architecture board review?

Material risk, unusual trade-offs, cross-platform impact, and exceptions - not every standard deployment.

Chapter Summary

- Translate obligations into testable outcomes.
- Layer preventive, detective, and corrective controls.
- Engineer evidence deliberately.
- Govern exceptions with ownership and expiry.

Chapter 68 - Google Cloud Certification Mastery

Building a role-based, evidence-driven preparation plan for current Google Cloud certifications

Learning Objectives

- Select certification paths aligned to job role and current experience.
- Use exam guides, skills inventory, labs, and scenario practice rather than memorization.
- Develop a gap-based study plan with measurable evidence.
- Integrate certification preparation with portfolio and workplace capability.

68.1 Role-Based Certification Paths

Google Cloud offers foundational, associate, and professional certifications. Common engineering paths include Associate Cloud Engineer, Professional Cloud Architect, Professional Cloud DevOps Engineer, Professional Cloud Security Engineer, Professional Cloud Network Engineer, Professional Data Engineer, and Professional Machine Learning Engineer.

Certification names, exam guides, product coverage, renewal options, languages, and policies change. Always use the current official exam guide and sample questions rather than relying on an old course outline.

68.2 Gap-Based Study Method

Create a domain-by-domain skills matrix: explain, design, implement, troubleshoot, optimize, and communicate. Mark evidence from labs, incidents, projects, and documentation. Study the highest-impact gaps rather than rereading familiar material.

Use retrieval practice, spaced repetition, scenario comparison, architecture diagrams, command practice, and timed question review. Build a wrong-answer log that records why each alternative was wrong.

68.3 Exam Technique

Read for business requirement, constraint, stage of migration, operational maturity, and the question verb. Eliminate options that violate stated constraints, create unnecessary operations, weaken security, or ignore recovery and cost.

Professional exams often reward trade-off reasoning. Choose the solution that best satisfies the scenario, not the most feature-rich product. Flag uncertain questions and manage time deliberately.

68.4 Portfolio and Career Integration

Certification is stronger when paired with evidence: architecture reviews, migration plans, Terraform, secure pipelines, observability, incident simulations, cost analysis, and capstones. Explain what you built, what failed, what you measured, and what you changed.

Use certification preparation to close real operating gaps. The goal is durable engineering capability, with the credential as external validation.

Certification principle: memorize terminology only where necessary; invest most effort in scenario judgment, hands-on evidence, and explaining trade-offs.

Production Design Checklist

- Current official exam guide is the source of truth.
- Study plan is based on measured gaps.
- Hands-on labs cover design, operations, and troubleshooting.
- Wrong-answer log explains reasoning.
- Portfolio evidence supports the claimed role.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Practice score plateaus	Review repeats facts without diagnosing reasoning gaps	Classify errors by domain and decision pattern	Weak domains improve in timed mixed sets
Labs do not improve exam judgment	Exercises are command-only without scenarios	Add constraints, failures, validation, and design explanation	Candidate can compare alternatives from evidence
Outdated course conflicts with exam	Product coverage changed	Return to current exam guide and release notes	Study map matches official domains
Exam anxiety causes time loss	No timed practice or question strategy	Use timed blocks, flagging, and breathing routine	Full practice completes with review time

Hands-on Lab 68: Create a 90-Day Google Cloud Certification Plan

Prerequisites

- A target certification and access to its current official exam guide.

Implementation

1. Map every exam domain to explain, design, implement, troubleshoot, optimize, and communicate skills.
2. Score confidence and attach evidence from Volumes 8A-8F.
3. Select the three largest gaps and define labs or projects that produce evidence.
4. Create a weekly plan with documentation study, retrieval practice, labs, and timed scenarios.
5. Maintain a wrong-answer log and update the skills matrix every week.
6. Run two full timed practice sessions and conduct a gap review.
7. Schedule the exam only when evidence and timed performance meet the defined threshold.

Validation Evidence

- Every domain has evidence or a remediation plan.
- Study time focuses on weak areas.
- Timed practice is completed.
- Current exam guide is referenced.

Cleanup

- Remove personal exam registration or payment details from shared artifacts.
- Retain the sanitized skills matrix and study log.

Interview Questions and Model Answers

1. Why use the exam guide?

It is the current authoritative map of assessed domains and product coverage.

2. What is retrieval practice?

Actively recalling and applying knowledge without looking at notes.

3. How should wrong answers be reviewed?

Record the misunderstood requirement, why the chosen option failed, and why the correct trade-off fits.

4. Does certification prove production experience?

No; it validates knowledge and judgment against an exam and should be paired with hands-on evidence.

Chapter Summary

- Choose certifications by role.
- Study from measured gaps.
- Practice scenarios and operations.
- Pair credentials with portfolio evidence.

Chapter 69 - Google Cloud Technical Interview and System Design Mastery

Communicating architecture, operations, incidents, and trade-offs clearly under interview pressure

Learning Objectives

- Use structured answer formats for behavioral, troubleshooting, and architecture questions.
- Run a repeatable Google Cloud system-design process.
- Demonstrate depth without bluffing and clarify assumptions.
- Build a story bank from labs, projects, incidents, and decisions.

69.1 Answer Structures

For behavioral questions, use situation, task, action, result, and learning. Quantify impact and separate your contribution from the team. For troubleshooting, use impact, scope, evidence, hypotheses, safe tests, mitigation, recovery, and prevention.

For architecture questions, restate goals and constraints, estimate scale, identify data and security boundaries, propose a high-level design, walk through critical paths, failure modes, operations, cost, and trade-offs.

69.2 Google Cloud System Design Flow

Clarify users, traffic, latency, consistency, availability, RTO/RPO, data classification, region, compliance, budget, team skill, and migration constraints. Draw boundaries before naming services.

Choose global ingress, compute, data, messaging, identity, secrets, networking, observability, deployment, and recovery. Explain why each choice fits and name at least one credible alternative.

69.3 Depth, Honesty, and Follow-up

Interviewers test reasoning by changing constraints. State assumptions and update the design when they change. When you do not know a product detail, say what you know, identify the risk, and explain how you would verify it.

Depth includes failure behavior, quotas, IAM, data lifecycle, rollout, on-call, cost, and migration - not only a clean diagram.

69.4 Story Bank and Portfolio

Build stories for difficult migration, production incident, security improvement, automation, cost reduction, performance bottleneck, disagreement, failed experiment, mentoring, and architecture trade-off.

Each story should include evidence: before and after metrics, diagrams, code, decision record, test result, or lesson. Never disclose employer-sensitive information.

Interview principle: strong candidates make uncertainty visible, ask clarifying questions, and explain why a design is appropriate - not merely what products they remember.

Production Design Checklist

- Requirements are clarified before design.
- Scale and failure assumptions are stated.
- Security, data, operations, and cost are included.
- Alternatives and trade-offs are explained.
- Stories contain measurable results and learning.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Answer becomes a service list	Requirements and boundaries were skipped	Restart with goals, constraints, and critical paths	Design choices map to stated requirements
Candidate overclaims knowledge	Uncertainty feels unsafe	State limits and verification method	Answer remains credible and technically useful
Diagram ignores operations	Focus is only happy-path architecture	Add deployment, telemetry, incident, backup, and ownership	Design can be operated and recovered
Behavioral story is vague	No result or personal contribution	Use STAR plus learning and metrics	Interviewer can identify action and impact

Hands-on Lab 69: Run a Senior Google Cloud Mock Interview

Prerequisites

- Two participants, a system-design prompt, a troubleshooting prompt, and a behavioral question set.

Implementation

1. Select a scenario such as global SaaS, data platform, event system, or migration.
2. Spend five minutes clarifying goals, constraints, scale, and failure objectives.
3. Design the system and explain critical request, data, deployment, and recovery paths.
4. Have the interviewer change one constraint and revise the design.
5. Run a troubleshooting scenario using evidence and hypotheses.
6. Answer three behavioral questions from the story bank.
7. Score clarity, depth, correctness, trade-offs, honesty, and time management; repeat the weakest section.

Validation Evidence

- Assumptions are explicit.
- Design includes security and operations.
- Alternatives are credible.
- Feedback leads to a second improved attempt.

Cleanup

- Delete recordings if consent or retention does not allow storage.
- Sanitize employer details in story notes.

Interview Questions and Model Answers

1. What should happen before naming products?

Clarify outcomes, constraints, scale, data, security, reliability, and operating model.

2. How do you handle an unknown detail?

State what you know, identify the decision risk, and explain how you would verify it.

3. What makes a good trade-off answer?

It compares alternatives against the scenario and explains consequences.

4. Why include operations in system design?

A design is incomplete if it cannot be deployed, observed, recovered, secured, and owned.

Chapter Summary

- Use structured answers.
- Clarify before designing.
- Explain alternatives and uncertainty.
- Support claims with evidence and stories.

Chapter 70 - Final Integrated Enterprise Google Cloud Capstone

Designing, delivering, operating, and defending a portfolio-grade enterprise platform

Learning Objectives

- Integrate all Volume 8 capabilities into one governed enterprise solution.
- Produce architecture, code, pipelines, telemetry, security, cost, recovery, and operational evidence.
- Demonstrate migration, incident, failover, performance, and governance readiness.
- Present the system as a portfolio case study and technical defense.

70.1 Capstone Scenario

A regulated digital-services company is modernizing from two data centers and one legacy cloud account. It serves business customers in multiple regions, processes transactional and analytical data, exposes public APIs, and is adding an AI-assisted support feature. The company requires strong tenant isolation, controlled data residency, regional recovery, secure software supply chain, centralized observability, audit evidence, and measurable unit economics.

The initial target supports one primary and one recovery region, with a path to regional cells. Teams need self-service delivery while the central platform team maintains identity, network, policy, security, telemetry, and cost guardrails.

70.2 Required Architecture

Design organization, folders, projects, billing, IAM, workload federation, Shared VPC, service perimeters, private access, DNS, global load balancing, Cloud Armor, compute, data, messaging, secrets, encryption, observability, CI/CD, and recovery.

Use at least two workload classes, such as Cloud Run for stateless APIs and GKE for specialized services. Include Cloud SQL, AlloyDB, Spanner, Firestore, or BigQuery based on explicit data requirements, plus Pub/Sub or Eventarc for asynchronous work.

70.3 Required Evidence

Produce requirements, quality-attribute scenarios, diagrams, ADRs, threat model, IAM matrix, data flow, Terraform or Infrastructure Manager deployment, CI/CD provenance, policy tests, dashboards, SLOs, restore results, failover timeline, incident simulation, cost model, and operational acceptance.

Every claim must link to evidence. A diagram without deployment or a runbook without a successful exercise is incomplete.

70.4 Delivery Phases

Phase 1 establishes governance and landing zone. Phase 2 deploys secure shared services and observability. Phase 3 builds application and data paths. Phase 4 adds delivery automation and security gates. Phase 5 validates performance, recovery, incident response, cost, and compliance. Phase 6 presents and defends the design.

Use incremental acceptance criteria and keep the system deployable after every phase. Record defects and rebuild until evidence meets the graduation standard.

70.5 Graduation Standard

The capstone graduates when a reviewer can deploy or inspect the configuration, trace one user journey, verify identity and data boundaries, observe health, understand cost, execute recovery, review incident evidence, and explain the major trade-offs.

The final presentation should include business context, architecture, risk decisions, demonstrated operations, defects discovered, improvements made, and next roadmap.

Capstone principle: the deliverable is not the cloud environment alone; it is the environment plus the evidence that it can be changed, observed, secured, recovered, governed, and explained.

Production Design Checklist

- Requirements and ADRs are complete.
- Deployment is automated and reproducible.
- Identity, network, data, and supply-chain controls are tested.
- SLO, performance, failover, restore, and incident evidence exist.
- Cost, ownership, runbooks, and roadmap are documented.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Capstone is only a diagram	Implementation and evidence were deferred	Select a smaller deployable scope and produce proof	Reviewer can reproduce the core environment
Deployment works only for author	Manual prerequisites or hidden credentials	Document bootstrap and use federation and automation	Independent reviewer deploys successfully
Recovery claim is theoretical	Failover or restore was not run	Execute controlled exercise and record timeline	Observed RTO/RPO and data checks are available
Presentation cannot explain trade-offs	Products were selected without alternatives	Create ADRs and business-linked decision narrative	Reviewer understands why choices fit the scenario

Hands-on Lab 70: Complete the Enterprise Google Cloud Graduation Project

Prerequisites

- A sandbox organization or project structure, budget, approved services, source repository, and sufficient time to iterate through defects.

Implementation

1. Write the business brief, constraints, quality attributes, acceptance criteria, and risk register.
2. Design organization, project, IAM, network, security, data, workload, delivery, telemetry, cost, and recovery architecture.
3. Implement the foundation and workloads through versioned infrastructure as code and keyless CI/CD.
4. Create dashboards, SLOs, audit evidence, vulnerability and policy gates, and operational runbooks.
5. Run performance, restore, regional failure, compromised identity, and cost-review exercises.
6. Fix defects and repeat tests until acceptance criteria are met.
7. Publish a sanitized portfolio package and conduct a technical defense with questions and trade-off challenges.

Validation Evidence

- Independent deployment or inspection succeeds.
- Critical user journey is observable end to end.
- Recovery and security exercises produce evidence.
- All high risks are remediated or accepted by an owner.

Cleanup

- Destroy paid sandbox resources after preserving sanitized evidence.
- Revoke temporary identities, tokens, and reviewer access.

Interview Questions and Model Answers

1. What is the minimum viable capstone?

A small but end-to-end system that is automated, secured, observable, recoverable, costed, and supported by evidence.

2. Why include defects?

Discovering and correcting defects demonstrates engineering learning and operational maturity.

3. What should be presented to an interviewer?

Business context, architecture, decisions, implementation, measured outcomes, failures, improvements, and personal contribution.

4. When is the platform complete?

When it meets explicit acceptance criteria and can be operated and recovered by someone other than the author.

Chapter Summary

- Integrate architecture and operations.
- Make every claim evidence-backed.
- Iterate through defects and retest.
- Present the work as a business and engineering story.

Part F Command and Investigation Reference

Use this reference as a starting point. Commands require appropriate project, region, permissions, and safety review. Prefer read-only inspection before mutation during incidents.

Purpose	Example Command	Operational Note
Active configuration	<code>gcloud config list</code>	Confirm account, project, region, and zone before action.
Project inventory	<code>gcloud projects list</code>	Use filters and organization scope; avoid assuming display name is unique.
IAM policy	<code>gcloud projects get-iam-policy PROJECT_ID --format=json</code>	Preserve output before changing bindings.
Audit logs	<code>gcloud logging read 'protoPayload.methodName:*' --limit=50 --format=json</code>	Narrow time and resource scope to reduce noise and cost.
Compute instances	<code>gcloud compute instances list</code>	Check zone, status, service account, labels, and network.
MIG status	<code>gcloud compute instance-groups managed list</code>	Review current actions, target size, autoscaler, and versions.
GKE clusters	<code>gcloud container clusters list</code>	Confirm location, release channel, private settings, and fleet membership.
Cloud Run services	<code>gcloud run services list --platform=managed</code>	Inspect revision, traffic, ingress, service identity, and VPC egress.
Artifact images	<code>gcloud artifacts docker images list LOCATION-docker.pkg.dev/PROJECT/REPO</code>	Verify digest and provenance rather than mutable tag.
Pub/Sub backlog	<code>gcloud pubsub subscriptions describe SUBSCRIPTION</code>	Review retention, dead-letter policy, ordering, and exactly-once settings where used.
Cloud SQL	<code>gcloud sql instances describe INSTANCE</code>	Inspect HA, backups, maintenance, private IP, and flags.
Spanner	<code>gcloud spanner instances describe INSTANCE</code>	Review node or processing units, configuration, and autoscaling.
Storage inventory	<code>gcloud storage ls --long gs://BUCKET</code>	Do not expose object names or metadata in shared incident channels.
SCC findings	<code>gcloud scc findings list ORGANIZATION_ID --source=SOURCE_ID</code>	Use current SCC CLI syntax and permissions; export evidence before remediation.
Asset search	<code>gcloud asset search-all-resources --scope=organizations/ORG_ID</code>	Useful for blast-radius and ownership investigation.

Purpose	Example Command	Operational Note
Service health	Check Google Cloud Service Health and Personalized Service Health	Correlate provider events with workload evidence; do not assume causation.
Billing export	Query detailed billing export in BigQuery	Restrict access because labels and project names can reveal sensitive context.
Terraform plan	terraform plan -out=tfplan && terraform show -json tfplan	Review create, update, replace, and destroy actions before apply.
Log-based timeline	Export selected logs to a protected evidence project	Record query, time zone, hash, owner, and retention.

Production Readiness Graduation Checklist

Architecture and Ownership

- ☐ Business outcomes, constraints, owners, and quality attributes are documented.
- ☐ ADRs cover high-consequence choices and rejected alternatives.
- ☐ Service catalog, escalation, and support hours are current.

Identity and Security

- ☐ Human and workload access use least privilege and short-lived credentials.
- ☐ Break-glass access is controlled, logged, and tested.
- ☐ Threat model, vulnerability process, incident playbooks, and evidence paths exist.

Network and Data

- ☐ Ingress, egress, DNS, routing, private access, perimeters, and firewall boundaries are documented.
- ☐ Data classification, ownership, residency, retention, encryption, backup, and deletion are tested.

Delivery and Supply Chain

- ☐ Infrastructure and application changes are versioned, reviewed, scanned, attested, and progressively deployed.
- ☐ Rollback and forward-fix paths are tested.
- ☐ Mutable tags and long-lived deployment keys are prohibited.

Observability and Reliability

- ☐ Critical journeys have SLIs, SLOs, dashboards, actionable alerts, and error-budget policy.
- ☐ Restore, failover, capacity, dependency failure, and incident communication are exercised.

Cost and Governance

- ☐ Projects, labels, budgets, unit costs, quotas, commitments, and cleanup ownership are current.
- ☐ Policy exceptions have owners, compensating controls, expiry, and review.

Operational Acceptance

- ☐ Runbooks can be executed by someone other than the author.
- ☐ Known risks are remediated or formally accepted.
- ☐ Evidence is stored, protected, and reproducible.

Ten Additional Enterprise Portfolio Projects

1. Multi-Region SaaS Cell Platform

Build two tenant cells with global ingress, tenant routing, quotas, SLOs, cost allocation, and a tested cell evacuation.

- Deliverables: architecture diagram, ADRs, IaC, runbook, dashboard, test evidence, cost model, and five-minute portfolio narrative.

2. Regulated Landing Zone

Create organization policies, Shared VPC, logging aggregation, security findings, key management, billing export, and evidence pack.

- Deliverables: architecture diagram, ADRs, IaC, runbook, dashboard, test evidence, cost model, and five-minute portfolio narrative.

3. Migration Factory

Produce workload inventory, dependency mapping, rationalization, wave plan, cutover rehearsal, rollback, and hypercare metrics.

- Deliverables: architecture diagram, ADRs, IaC, runbook, dashboard, test evidence, cost model, and five-minute portfolio narrative.

4. Secure Software Supply Chain

Implement Artifact Registry, Cloud Build, provenance, vulnerability policy, Binary Authorization, and keyless delivery.

- Deliverables: architecture diagram, ADRs, IaC, runbook, dashboard, test evidence, cost model, and five-minute portfolio narrative.

5. Data Platform Recovery

Deploy a governed analytical pipeline and demonstrate replay, schema failure handling, backup, restore, and cost controls.

- Deliverables: architecture diagram, ADRs, IaC, runbook, dashboard, test evidence, cost model, and five-minute portfolio narrative.

6. Platform Engineering Golden Path

Offer self-service Cloud Run and GKE paths with Terraform modules, CI/CD, identity, telemetry, policy, and exception workflow.

- Deliverables: architecture diagram, ADRs, IaC, runbook, dashboard, test evidence, cost model, and five-minute portfolio narrative.

7. Incident Response Evidence Lab

Simulate identity compromise, preserve audit evidence, contain, rebuild, rotate, detect persistence, and publish a retrospective.

- Deliverables: architecture diagram, ADRs, IaC, runbook, dashboard, test evidence, cost model, and five-minute portfolio narrative.

8. Performance and Capacity Qualification

Create workload model, test suite, bottleneck analysis, failover capacity, quotas, and cost-per-transaction report.

- Deliverables: architecture diagram, ADRs, IaC, runbook, dashboard, test evidence, cost model, and five-minute portfolio narrative.

9. FinOps Control Tower

Use billing export, allocation, anomaly detection, budgets, commitment analysis, optimization backlog, and realized-savings validation.

- Deliverables: architecture diagram, ADRs, IaC, runbook, dashboard, test evidence, cost model, and five-minute portfolio narrative.

10. AI-Enabled Support Platform

Build tenant-scoped retrieval, model evaluation, prompt-injection controls, tool authorization, safety monitoring, and unit-cost limits.

- Deliverables: architecture diagram, ADRs, IaC, runbook, dashboard, test evidence, cost model, and five-minute portfolio narrative.

90-Day Google Cloud Engineering Practice Plan

Weeks	Primary Focus	Evidence Outcome
1-2	Architecture and Well-Architected review	Requirements, ADRs, risk register, review evidence
3-4	Migration and modernization	Inventory, wave plan, decomposition exercise, reconciliation
5-6	Global reliability and performance	Failover, chaos experiment, load qualification, capacity plan
7-8	Security and governance	Incident simulation, evidence pack, policy exception
9-10	FinOps and platform engineering	Unit economics, optimization, golden path prototype
11	Certification and interview drills	Gap matrix, timed scenarios, mock interviews
12-13	Final capstone	Deploy, test, fix, retest, present, and publish portfolio

- Practice at least five days each week: two design sessions, two hands-on sessions, and one review or interview session.
- Keep an engineering journal with assumptions, commands, errors, hypotheses, evidence, and lessons.
- Repeat any lab that produced unexplained behavior; do not hide failed attempts.
- Publish only sanitized artifacts that do not expose credentials, customer data, internal hostnames, or employer-sensitive details.

Glossary

Term	Definition
ADR	Architecture Decision Record: context, decision, alternatives, consequences, status, and review trigger.
Blast radius	The users, systems, data, or business processes affected by a change or failure.
Cell architecture	A topology that partitions workloads or tenants into independently operable units to limit correlated failure.
Control objective	The risk-reducing outcome that a control is expected to achieve.
Error budget	The allowed unreliability derived from an SLO over a defined period.
Evidence engineering	Designing evidence collection, integrity, retention, access, and reproducibility as part of the system.
Failback	The controlled return from a recovery topology to the preferred topology.
Golden path	A supported, reusable, secure, observable, and operable route for a common workload class.
Idempotency	The property that repeated processing of the same request or event does not create unintended additional effects.
Migration factory	A repeatable operating model for assessing, moving, validating, and optimizing groups of workloads.
RPO	Recovery Point Objective: the maximum acceptable data-loss window.
RTO	Recovery Time Objective: the maximum acceptable time to restore the required business capability.
SLI	Service Level Indicator: a measured signal representing service behavior.
SLO	Service Level Objective: a target for an SLI over a period.
Steady state	A measurable description of normal user and system behavior used in resilience experiments.
Tenant isolation	Controls that prevent one SaaS tenant from accessing or degrading another tenant's resources or data.
Unit economics	The cost to deliver a measurable product outcome such as a transaction, tenant, model request, or active user.
Well-Architected review	A structured evaluation of workload risks, evidence, and trade-offs across Google Cloud architecture pillars.

Official Google Cloud Documentation Index

Reference	Official URL
Google Cloud Well-Architected Framework	https://docs.cloud.google.com/architecture/framework
Reliability pillar	https://docs.cloud.google.com/architecture/framework/reliability
Operational excellence - incident management	https://docs.cloud.google.com/architecture/framework/operational-excellence/manage-incidents-and-problems
Migration to Google Cloud	https://docs.cloud.google.com/architecture/migrations
Migration Center and RaMP	https://docs.cloud.google.com/migration-center/docs/ramp-overview
Disaster recovery planning guide	https://docs.cloud.google.com/architecture/dr-scenarios-planning-guide
Architecting disaster recovery	https://docs.cloud.google.com/architecture/disaster-recovery
Google Cloud certification	https://cloud.google.com/learn/certification
Professional Cloud Architect	https://cloud.google.com/learn/certification/cloud-architect
Professional Cloud DevOps Engineer	https://cloud.google.com/learn/certification/cloud-devops-engineer
Associate Cloud Engineer	https://cloud.google.com/learn/certification/cloud-engineer
Cloud Architecture Center	https://docs.cloud.google.com/architecture
Google Cloud security documentation	https://docs.cloud.google.com/docs/security
Cloud Audit Logs	https://docs.cloud.google.com/logging/docs/audit
Cloud Monitoring	https://docs.cloud.google.com/monitoring/docs
Cloud Logging	https://docs.cloud.google.com/logging/docs
Security Command Center	https://docs.cloud.google.com/security-command-center/docs
Cloud Asset Inventory	https://docs.cloud.google.com/asset-inventory/docs
VPC Service Controls	https://docs.cloud.google.com/vpc-service-controls/docs
Cloud Billing export	https://docs.cloud.google.com/billing/docs/how-to/export-data-bigquery
FinOps on Google Cloud	https://cloud.google.com/finops
Google Cloud release notes	https://docs.cloud.google.com/release-notes

Product behavior, regional availability, quotas, pricing, preview status, and exam content change. Validate current official documentation and release notes before implementation or exam registration.

Volume 8 Completion

With Part F complete, Volume 8 now covers Google Cloud from foundations through enterprise architecture, secure delivery, data platforms, cloud-native applications, operations, governance, certification, interviews, and portfolio-grade capstones.

Next volume in the DevOps Master Course roadmap: Volume 9 - Terraform and Ansible Infrastructure Automation.